BetterUp

PEOPLE TECHNOLOGY · TAKE-HOME EXERCISE

AI Automation Engineer

A real BetterUp onboarding problem that spans our People systems. We're looking for engineering depth — reasoning across systems and writing the code that ties them together.

TIME NEEDED	SUBMIT TO	FORMAT
About 2 hours	**jessica.brewer@betterup.co**	**Doc, repo, or Loom · 24h before debrief**

Context

This role does the heavy technical work across our People systems that low-code alone can't reach. This exercise gives you a real problem that touches several systems at once — we care less about how much you finish than how you reason across systems and the engineering choices you make. Use AI freely as you build; the prototype itself should be real code you can walk us through.

The problem

BetterUp is hiring fast, and onboarding one hire means keeping data in sync across Ashby, Workday, Okta/Lumos, equipment, and a cohort tracker. Today that coordination is manual and brittle:

- **Changes don't propagate.** When a start date, name, or address changes after the offer — or an off-cycle hire is created outside Ashby — the update has to ripple across Workday, Okta/Lumos, equipment, and the tracker. Today it's manual, undocumented, and it breaks.
- **Bad data cascades.** A wrong name or location in Ashby flows straight into Workday, the Okta email, shipping labels, and the tracker — with no cross-system check to catch it.
- **Nobody's watching the gates.** The IT hardware form, background check, and Okta provisioning all have deadlines (often 7 days out). Today someone chases them by hand, if at all.

Workday · Ashby · Okta · Lumos · Google Workspace · Slack · Highspot · ExpoIT · n8n · Zapier · Claude · Claude Code · Cursor

How we'll evaluate it

- ✓ Cross-system, cross-process thinking — not a one-off automation
- ✓ Engineering depth — real code, sound data flow, failure handling, idempotency
- ✓ Claude used as a meaningful component, not the entire solution
- ✓ Full ownership — you can defend every technical decision

Optional bonus, if you have time: an MCP tool definition, an authentication approach (OAuth/SSO), or an audit trail.

The ask

Pick the highest-leverage slice — change propagation, cross-system data validation, or exception monitoring — and build a working prototype of its core logic.

1. Choose the slice with the most leverage, and defend why over the others — that choice is part of the test.
2. Write the real logic — the connector, transformation, or orchestration that moves and reconciles data across systems, with error handling. Build against mocked APIs or sample data; no real credentials needed.
3. Where it fits, use Claude as a component — an agentic decision or exception handler via API/MCP — not as the whole solution.

Use whatever helps you build — Claude, Claude Code, Cowork, Cursor. But the deliverable is code you can walk us through and defend: we want to see you engineer across systems, not just wire up an assistant.

One constraint

Build with the tools we already have.
A big part of this role is getting more out of what we've already invested in, rather than adding new tools. Build with our existing stack. If you truly believe a new tool is needed, name it and make the case for why our current stack can't do the job — we're just as interested in what you'd choose *not* to add.

Deliverables

1. **A quick systems map.** Where the data lives and moves, where it breaks today, and the slice you chose to build — and why. (~20 min.)
2. **A working, runnable prototype.** Real code, demoable on sample/mocked data. Include the Claude/MCP piece if you use one. A repo or short Loom is fine.
3. **A short build note.** Why this slice, your key design decisions (data model, failure handling, idempotency), what you'd productionize, and which AI tools you used — including one thing the AI got wrong and how you caught it.